

Documentation Technique

Objectif, outils utilisés et explication du code, bloc par bloc

Projets 1 à 7 — dossier Talent-Performer-Projects-main

Ce document couvre les projets numérotés 1 à 7 du dépôt. Le dossier **6** n'existe pas dans le dépôt (la numérotation passe directement de 5 à 7) : ce document en couvre donc **6**, pas 7.

Sommaire

#	Projet	En une phrase
1	1-Agno	Assistant Q&A multilingue (FR/AR/EN) sur des articles collés ou PDF, avec export PDF/Word.
2	2-assembyai	Transcription audio (Whisper) + résumé/explication IA, en anglais, arabe ou darija.
3	3-Graphiti	Suivi d'activités sportives dans un graphe Neo4j (+ version simulée hors-ligne).
4	4-FireCrawl_PDFparsing	Extraction d'un PDF en ligne (Firecrawl) puis questions/réponses dessus.
5	5-FireCrawl_Scrape	Scraper universel d'annonces (immobilier, véhicules, emploi...) avec analyse IA.
7	7-UQLM	Détecteur d'hallucinations : mesure l'incertitude d'une réponse IA avant de l'accepter.

Projet 1 — 1-Agno

Assistant multilingue de questions/réponses sur des articles (RAG)

Objectif

Permettre à l'utilisateur de coller un texte ou d'importer un article (PDF ou .txt), dans n'importe quelle langue — français, arabe ou anglais — puis de lui poser des questions en langage naturel et d'obtenir une réponse fiable, basée uniquement sur le contenu fourni (technique RAG : *Retrieval-Augmented Generation*). La conversation peut ensuite être exportée en PDF ou Word, avec une mise en forme correcte du texte arabe (lettres liées, sens droite-à-gauche).

Outils utilisés

Outil	Rôle dans ce projet
Streamlit	Construit l'interface web (barre latérale, chat, boutons de téléchargement).
Agno	Framework qui orchestre l'agent conversationnel et le relie à la fonction de recherche maison.
Groq (Llama / gpt-oss-120b)	Modèle de langage qui rédige les réponses, rapide et multilingue.
sentence-transformers	Transforme chaque passage de texte en vecteur numérique (modèle multilingue, ~50 langues).
FAISS	Index vectoriel qui retrouve les passages les plus proches d'une question en mémoire.
PyMuPDF	Extrait le texte des PDF importés (rapide).
python-docx / fpdf2	Génèrent respectivement les exports Word et PDF de la conversation.
arabic-reshaper / python-bidi	Mettent en forme le texte arabe correctement dans les exports.

rag_engine.py

Le moteur : extraction PDF, découpage du texte, recherche par similarité, agent IA.

Lignes 1-15

```
import os
import re
from typing import Optional

import faiss
import numpy as np
import pymupdf
from dotenv import load_dotenv
from sentence_transformers import SentenceTransformer

from agno.agent import Agent
from agno.models.groq import Groq

load_dotenv()
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
```

Importe les bibliothèques nécessaires : **faiss** pour l'index vectoriel, **numpy** pour les tableaux numériques, **pymupdf** pour lire les PDF, **sentence_transformers** pour créer les embeddings, et **agno** pour l'agent conversationnel relié à Groq. `load_dotenv()` charge le fichier `.env` et récupère la clé API Groq dans une variable globale.

Lignes 17-22

```
EMBEDDING_MODEL = "paraphrase-multilingual-MiniLM-L12-v2"
MIN_CHUNK_LEN = 40
CHUNK_SIZE = 900
CHUNK_OVERLAP = 150
```

Constantes de configuration : le nom du modèle d'embeddings multilingue (couvre le français, l'arabe, l'anglais et ~50 autres langues), la longueur minimale d'un passage pour être indexé (évite les fragments inutiles), et la taille/chevauchement des blocs de découpage du texte.

Lignes 25-26

```
def load_embedder() -> SentenceTransformer:
    return SentenceTransformer(EMBEDDING_MODEL)
```

Charge le modèle d'embeddings depuis Hugging Face. Cette fonction est appelée une seule fois par session grâce à la mise en cache côté interface (`app.py`).

Lignes 29-33

```
def extract_text_from_pdf(file) -> str:
    # PyMuPDF est ~20x plus rapide que pypdf pour extraire le texte d'un PDF.
    data = file.read() if hasattr(file, "read") else file
    with pymupdf.open(stream=data, filetype="pdf") as doc:
        return "\n\n".join(page.get_text() for page in doc)
```

Extrait le texte brut d'un PDF. Accepte aussi bien un fichier uploadé par Streamlit (qui a une méthode `.read()`) que des octets bruts. Chaque page est jointe avec une ligne vide entre elles.

Lignes 36-64

```
def chunk_text(text: str, source: str) -> list[dict]:
    normalized = re.sub(r"\s+", " ", text).strip()
    if not normalized:
        return []

    chunks = []
    start = 0
    n = len(normalized)
    while start < n:
        end = min(start + CHUNK_SIZE, n)
        if end < n:
            boundary = normalized.rfind(" ", start, end)
            if boundary > start:
                end = boundary
        piece = normalized[start:end].strip()
        if len(piece) >= MIN_CHUNK_LEN:
            chunks.append({"content": piece, "source": source})
        if end >= n:
            break
        start = max(end - CHUNK_OVERLAP, start + 1)
    return chunks
```

Découpe un texte en blocs de taille régulière avec chevauchement, au lieu de couper naïvement sur chaque retour à la ligne (ce qui produisait des centaines de micro-fragments pour un PDF, très lents à encoder). `re.sub(r"\s+", " ", text)` aplatit d'abord tous les espaces/retours à la ligne en un seul espace. La boucle `while` avance ensuite par fenêtre de `CHUNK_SIZE` caractères, en reculant jusqu'au dernier espace (`rfind`) pour ne jamais couper un mot en deux, puis recule de `CHUNK_OVERLAP` caractères avant de continuer, pour garder du contexte entre deux blocs consécutifs.

Lignes 67-73

```
class KnowledgeBase:
    """Base de connaissances multilingue en mémoire, ré-indexable à la volée."""

    def __init__(self, embedder: SentenceTransformer):
        self.embedder = embedder
        self.chunks: list[dict] = []
        self.index: Optional[faiss.IndexFlatL2] = None
```

La classe qui représente toute la mémoire de l'application : la liste des blocs de texte (`self.chunks`) et l'index vectoriel FAISS associé (`self.index`), initialement vide.

Lignes 75-88

```
def add_document(self, text: str, source: str) -> int:
    new_chunks = chunk_text(text, source)
    if not new_chunks:
        return 0
    embeddings = self.embedder.encode(
        [c["content"] for c in new_chunks], convert_to_numpy=True, show_progress_bar=False
    )
    if self.index is None:
        self.index = faiss.IndexFlatL2(embeddings.shape[1])
    self.index.add(embeddings)
    self.chunks.extend(new_chunks)
    return len(new_chunks)
```

Ajoute un nouvel article à la base : découpe le texte, encode **uniquement les nouveaux blocs** (pas toute la base à chaque fois — optimisation clé pour la rapidité), crée l'index FAISS au premier appel, puis ajoute les nouveaux vecteurs à l'index existant via `index.add()`.

Lignes 90-105

```

def remove_source(self, source: str) -> None:
    self.chunks = [c for c in self.chunks if c["source"] != source]
    self._rebuild_index()

def clear(self) -> None:
    self.chunks = []
    self.index = None

def _rebuild_index(self) -> None:
    if not self.chunks:
        self.index = None
        return
    texts = [c["content"] for c in self.chunks]
    embeddings = self.embedder.encode(texts, convert_to_numpy=True, show_progress_bar=False)
    self.index = faiss.IndexFlatL2(embeddings.shape[1])
    self.index.add(embeddings)

```

`remove_source()` retire tous les blocs d'un article donné puis reconstruit l'index en entier (opération rare, donc le coût est acceptable). `clear()` vide complètement la base. `_rebuild_index()` ré-encode tous les blocs restants et recrée un index FAISS neuf.

Lignes 107-118

```

def search(self, query: str, num_documents: int = 4) -> list[dict]:
    if not query or not query.strip() or self.index is None or not self.chunks:
        return []
    num_docs = min(num_documents, len(self.chunks))
    query_vec = self.embedder.encode([query], convert_to_numpy=True)
    _, indices = self.index.search(np.array(query_vec), num_docs)
    results = []
    for idx in indices[0]:
        if 0 <= idx < len(self.chunks):
            c = self.chunks[idx]
            results.append({"content": c["content"], "meta_data": {"source": c["source"]}})
    return results

```

Retrouve les blocs les plus pertinents pour une question : encode la question en vecteur, puis `index.search()` retourne les indices des `num_docs` vecteurs les plus proches (distance euclidienne). Ces indices sont ensuite reliés aux blocs de texte d'origine.

Lignes 121-137

```

def build_agent(knowledge_base: KnowledgeBase) -> Agent:
    def retriever(agent, query, num_documents=4, **kwargs):
        if num_documents is None:
            num_documents = 4
        return knowledge_base.search(query, num_documents)

    return Agent(
        model=Groq(id="openai/gpt-oss-120b", api_key=GRQO_API_KEY),
        knowledge=None,
        search_knowledge=True,
        knowledge_retriever=retriever,
        instructions=[
            "Tu es un assistant qui répond aux questions en te basant uniquement sur les articles",
            "fournis...",
            "Réponds toujours dans la même langue que la question posée (français, arabe ou",
            "anglais).",
            "Si l'information n'est pas présente..., dis-le clairement au lieu d'inventer une",
            "réponse.",
        ],
    )

```

Construit l'agent Agno. La fonction interne `retriever` est branchée sur `knowledge_retriever` : à chaque question, Agno l'appelle automatiquement pour récupérer les passages pertinents via `knowledge_base.search()`, qu'il injecte ensuite dans le contexte du modèle. Les instructions obligent l'agent à rester fidèle aux articles et à répondre dans la langue de la question.

L'interface Streamlit : barre latérale (articles) + fil de discussion.

Lignes 1-20

```
import streamlit as st

from export_chat import build_docx, build_pdf
from rag_engine import GROQ_API_KEY, KnowledgeBase, build_agent, extract_text_from_pdf,
load_embedder

st.set_page_config(page_title="Assistant Articles Multilingue", page_icon="")

st.markdown(
    """
    <style>
    .stChatMessage p, .stMarkdown p { direction: auto; unicode-bidi: plaintext; }
    </style>
    """,
    unsafe_allow_html=True,
)
```

Importe Streamlit et les fonctions des deux autres fichiers. `st.markdown` injecte une règle CSS qui laisse le navigateur détecter automatiquement le sens d'écriture (gauche-à-droite ou droite-à-gauche) de chaque message affiché, indispensable pour que l'arabe s'affiche correctement dans le chat.

Lignes 18-33

```
@st.cache_resource(show_spinner="Chargement du modèle d'embeddings...")
def get_embedder():
    return load_embedder()

def get_knowledge_base() -> KnowledgeBase:
    if "kb" not in st.session_state:
        st.session_state.kb = KnowledgeBase(get_embedder())
    return st.session_state.kb

def get_agent(kb: KnowledgeBase):
    if "agent" not in st.session_state or st.session_state.get("agent_kb") is not kb:
        st.session_state.agent = build_agent(kb)
        st.session_state.agent_kb = kb
    return st.session_state.agent
```

`@st.cache_resource` garantit que le modèle d'embeddings (lourd à charger) n'est créé qu'une seule fois pour toute la session, même si Streamlit ré-exécute le script à chaque clic. `get_knowledge_base()` et `get_agent()` utilisent `st.session_state` (le « presse-papier » persistant de Streamlit) pour ne créer la base de connaissances et l'agent qu'une seule fois également.

Lignes 36-52

```
def main():
    st.title("Assistant Articles - FR / EN / AR")
    st.caption(...)

    if not GROQ_API_KEY:
        st.error("GROQ_API_KEY est introuvable. Ajoute-le dans le fichier .env à la racine du projet.")
        st.stop()

    kb = get_knowledge_base()

    if "sources" not in st.session_state:
        st.session_state.sources = []
    if "messages" not in st.session_state:
        st.session_state.messages = []
```

Début de la fonction principale : titre, vérification que la clé API est bien configurée (sinon `st.stop()` arrête l'exécution du script avec un message d'erreur), récupération de la base de connaissances, puis initialisation des listes sources (noms des articles ajoutés) et messages (historique de conversation) dans `session_state` si elles n'existent pas encore.

Lignes 54-70

```
with st.sidebar:
    st.header(" Articles")

    pasted_text = st.text_area("Coller un article (n'importe quelle langue)", height=150)
    pasted_title = st.text_input("Titre de l'article collé", placeholder="ex: Article économie")
    if st.button("Ajouter le texte collé", use_container_width=True):
        if pasted_text.strip():
            title = pasted_title.strip() or f"Article collé #{len(st.session_state.sources) + 1}"

            with st.spinner("Indexation de l'article..."):
                added = kb.add_document(pasted_text, title)
            if added:
                st.session_state.sources.append(title)
                st.success(f"{added} passages ajoutés depuis « {title} ».")
            else:
                st.warning("Le texte collé est trop court pour être indexé.")
        else:
            st.warning("Colle d'abord un texte.")
```

Dans la barre latérale : une zone de texte pour coller un article et un champ pour son titre. Au clic sur le bouton, si du texte a bien été saisi, `kb.add_document()` l'indexe (avec un spinner pendant l'opération), et le titre est ajouté à la liste des sources affichées.

Lignes 72-87

```
uploaded_files = st.file_uploader(
    "Ou importer des fichiers (PDF / TXT)", type=["pdf", "txt"],
    accept_multiple_files=True
)
if uploaded_files and st.button("Ajouter les fichiers", use_container_width=True):
    for f in uploaded_files:
        if f.name in st.session_state.sources:
            continue
        with st.spinner(f"Indexation de « {f.name} »..."):
            if f.name.lower().endswith(".pdf"):
                text = extract_text_from_pdf(f)
            else:
                text = f.read().decode("utf-8", errors="ignore")
            added = kb.add_document(text, f.name)
        if added:
            st.session_state.sources.append(f.name)
            st.success(f"{added} passages ajoutés depuis « {f.name} ».")
```

Alternative à la saisie manuelle : import de fichiers PDF ou TXT (plusieurs à la fois). Pour chaque fichier non déjà indexé, extrait son texte (PDF via `extract_text_from_pdf`, TXT via simple décodage UTF-8) puis l'indexe de la même façon que le texte collé.

Lignes 89-104

```
if st.session_state.sources:
    st.divider()
    st.subheader("Articles indexés")
    for src in list(st.session_state.sources):
        col1, col2 = st.columns([4, 1])
        col1.write(f"• {src}")
        if col2.button("", key=f"del-{src}"):
            kb.remove_source(src)
            st.session_state.sources.remove(src)
            st.rerun()

    if st.button("Tout effacer", use_container_width=True):
        kb.clear()
        st.session_state.sources = []
        st.session_state.messages = []
        st.rerun()
```

Affiche la liste des articles déjà indexés, chacun avec un bouton (icone poubelle) pour le supprimer individuellement (`kb.remove_source`), plus un bouton global « Tout effacer » qui vide complètement la base et l'historique. `st.rerun()` force un rafraîchissement immédiat de la page après chaque modification.

Lignes 106-122

```
if st.session_state.messages:
    st.divider()
    st.subheader("Exporter la conversation")
    st.download_button(
        " Télécharger en PDF",
        data=build_pdf(st.session_state.messages, st.session_state.sources),
        file_name="conversation.pdf",
        mime="application/pdf",
        use_container_width=True,
    )
    st.download_button(
        " Télécharger en Word",
        data=build_docx(st.session_state.messages, st.session_state.sources),
        file_name="conversation.docx",
        ...
    )
```

Si une conversation existe, affiche deux boutons de téléchargement qui appellent respectivement `build_pdf()` et `build_docx()` (définies dans `export_chat.py`) pour générer le fichier à la volée, directement en mémoire (aucun fichier temporaire sur disque).

Lignes 124-136

```
if not st.session_state.sources:
    st.info(" Ajoute au moins un article dans la barre latérale pour commencer à poser des questions.")
    return

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

question = st.chat_input("Pose ta question en français, arabe ou anglais...")
if question:
    st.session_state.messages.append({"role": "user", "content": question})
    with st.chat_message("user"):
        st.markdown(question)
```

Si aucun article n'a encore été ajouté, affiche un message d'aide et arrête là (return). Sinon, réaffiche tout l'historique de conversation (bulles st.chat_message), puis capture une nouvelle question via st.chat_input (qui ne se déclenche qu'à l'appui sur Entrée) et l'ajoute immédiatement à l'historique.

Lignes 138-160

```
agent = get_agent(kb)
with st.chat_message("assistant"):
    with st.spinner("Recherche dans les articles..."):
        answer = None
        last_error = None
        for attempt in range(3):
            try:
                response = agent.run(question)
                answer = response.content
                break
            except Exception as e:
                last_error = e
        if answer is None:
            answer = (
                "Désolé, le modèle a rencontré une erreur technique... "
                f"Détail : {last_error}"
            )
    st.markdown(answer)
    st.session_state.messages.append({"role": "assistant", "content": answer})
    st.rerun()
```

Récupère l'agent puis l'interroge via agent.run(question), qui déclenche en interne le retriever (recherche FAISS) puis l'appel au modèle Groq. La boucle for attempt in range(3) réessaie jusqu'à 3 fois en cas d'erreur — Groq/Llama produit parfois un appel d'outil mal formé (erreur transitoire), un simple nouvel essai suffit presque toujours. La réponse est ensuite ajoutée à l'historique et st.rerun() rafraîchit la page (nécessaire pour que les boutons d'export apparaissent dès le premier message).

export_chat.py

Génère les fichiers PDF et Word téléchargeables de la conversation.

Lignes 1-23

```
import io
from datetime import datetime

import arabic_reshaper
from bidi.algorithm import get_display
from docx import Document
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.xml import OxmlElement
from fpdf import FPDF

ARABIC_RANGES = (
    (0x0600, 0x06FF), (0x0750, 0x077F), (0x08A0, 0x08FF), (0xFB50, 0xFDFF), (0xFE70, 0xFEFF),
)

PDF_FONT_CANDIDATES = [
    (r"C:\Windows\Fonts\tahoma.ttf", r"C:\Windows\Fonts\tahomabd.ttf"),
    (r"C:\Windows\Fonts\arialuni.ttf", r"C:\Windows\Fonts\arialuni.ttf"),
]
```

io permet de générer les fichiers directement en mémoire. arabic_reshaper et python-bidi gèrent la mise en forme de l'arabe. ARABIC_RANGES liste les plages de code Unicode couvrant les caractères arabes, utilisées pour détecter automatiquement si un texte en contient. PDF_FONT_CANDIDATES liste des polices Windows qui couvrent à la fois le latin et l'arabe, essayées dans l'ordre.

Lignes 26-38

```
def contains_arabic(text: str) -> bool:
    return any(any(lo <= ord(ch) <= hi for lo, hi in ARABIC_RANGES) for ch in text)

def _shape_line(line: str) -> str:
    if not line.strip():
        return ""
    return get_display(arabic_reshaper.reshape(line))

def _set_paragraph_rtl(paragraph) -> None:
    pPr = paragraph._p.get_or_add_pPr()
    pPr.append(OxmlElement("w:bidi"))
```

contains_arabic() teste si au moins un caractère du texte tombe dans les plages Unicode arabes. _shape_line() relie les lettres arabes entre elles puis les réordonne de droite à gauche (nécessaire pour le PDF, qui ne gère pas nativement le texte bidirectionnel). _set_paragraph_rtl() manipule directement le XML interne d'un paragraphe Word pour lui ajouter la balise w:bidi, qui active l'affichage droite-à-gauche natif de Word.

Lignes 41-72

```

def build_docx(messages: list[dict], sources: list[str]) -> bytes:
    doc = Document()
    doc.add_heading("Conversation – Assistant Articles", level=1)
    meta = doc.add_paragraph(f"Exporté le {datetime.now().strftime('%d/%m/%Y %H:%M')}")
    meta.runs[0].italic = True
    if sources:
        doc.add_paragraph("Articles utilisés : " + ", ".join(sources))
    doc.add_paragraph()

    for msg in messages:
        label = "Vous" if msg["role"] == "user" else "Assistant"
        content = msg["content"]
        is_rtl = contains_arabic(content)
        p_label = doc.add_paragraph()
        run_label = p_label.add_run(f"{label} :")
        run_label.bold = True
        p_content = doc.add_paragraph(content)
        if is_rtl:
            for paragraph in (p_label, p_content):
                _set_paragraph_rtl(paragraph)
                paragraph.alignment = WD_ALIGN_PARAGRAPH.RIGHT
                for run in paragraph.runs:
                    run.font.rtl = True

    buffer = io.BytesIO()
    doc.save(buffer)
    return buffer.getvalue()

```

Crée le document Word : titre, date d'export, liste des articles utilisés, puis chaque message (« Vous » ou « Assistant » en gras suivi du contenu). Pour chaque message contenant de l'arabe, applique l'alignement droite **et** la propriété `w:bidi` à la fois au paragraphe et à chacun de ses *runs* (segments de texte formatés). Le document final est sauvegardé dans un buffer mémoire (`io.BytesIO`) et ses octets sont retournés, prêts pour `st.download_button`.

Lignes 75-89

```

def _register_pdf_font(pdf: FPDF) -> str:
    for regular, bold in PDF_FONT_CANDIDATES:
        try:
            pdf.add_font("ChatFont", "", regular)
            pdf.add_font("ChatFont", "B", bold)
            return "ChatFont"
        except (RuntimeError, FileNotFoundError):
            continue
    return "helvetica"

def _write_line(pdf: FPDF, h: float, text: str, align: str = "L") -> None:
    pdf.multi_cell(0, h, text, align=align, new_x="LMARGIN", new_y="NEXT")

```

`_register_pdf_font()` essaie chaque police candidate jusqu'à en trouver une qui existe sur la machine, avec repli sur la police de base helvetica (sans support arabe) si aucune n'est trouvée. `_write_line()` est un petit correctif important : par défaut, `fpdf2` laisse le curseur collé à **droite** de la page après un `multi_cell`, ce qui épuise l'espace disponible ligne après ligne et finit par faire planter le PDF ; `new_x="LMARGIN", new_y="NEXT"` ramène systématiquement le curseur en marge gauche.

Lignes 92-116

```

def build_pdf(messages: list[dict], sources: list[str]) -> bytes:
    pdf = FPDF()
    pdf.add_page()
    font = _register_pdf_font(pdf)

    pdf.set_font(font, "B", 16)
    _write_line(pdf, 10, "Conversation - Assistant Articles", align="C")
    pdf.ln(2)

    pdf.set_font(font, "", 9)
    _write_line(pdf, 6, f"Exporté le {datetime.now().strftime('%d/%m/%Y %H:%M')}")
    if sources:
        _write_line(pdf, 6, "Articles utilisés : " + " ", ".join(sources))
    pdf.ln(4)

    for msg in messages:
        label = "Vous" if msg["role"] == "user" else "Assistant"
        pdf.set_font(font, "B", 11)
        _write_line(pdf, 7, _shape_line(f"{label} :"))
        pdf.set_font(font, "", 11)
        for line in msg["content"].split("\n"):
            _write_line(pdf, 7, _shape_line(line))
        pdf.ln(3)

    return bytes(pdf.output())

```

Équivalent PDF de `build_docx()` : titre centré, métadonnées, puis chaque message. Chaque ligne de texte passe par `_shape_line()` avant d'être écrite — sans effet sur du texte latin, mais indispensable pour l'arabe. `pdf.output()` sans argument retourne directement les octets du PDF (aucune écriture sur disque).

Projet 2 — 2-assembyai

Transcription audio et résumé/explication IA, multilingue (EN/AR/Darija)

Objectif

Permettre à l'utilisateur d'importer un fichier audio (.mp3/.wav), de le transcrire automatiquement, puis de demander à l'IA soit un **résumé**, soit une **explication détaillée** du contenu. L'interface est entièrement bilingue anglais/arabe (textes de l'interface eux-mêmes traduits selon la langue choisie), et prend en charge trois options de langue audio : anglais, arabe standard, et **darija** (arabe marocain dialectal). La transcription et la réponse IA peuvent chacune être exportées en PDF.

Outils utilisés

Outil	Rôle dans ce projet
Streamlit	Interface web : sélecteur de langue, import de fichier, zones de texte, boutons de téléchargement.
Groq (whisper-large-v3)	Modèle de transcription audio-vers-texte (speech-to-text), hébergé sur l'infrastructure rapide de Groq.
Groq (llama-3.3-70b-versatile)	Modèle de langage qui résume ou explique la transcription. modèle désormais retiré du catalogue Groq (voir remarque en fin de section).
fpdf2	Génère les PDF téléchargeables (transcription et réponse IA).
arabic-reshaper / python-bidi	Mettent en forme le texte arabe dans les PDF exportés.

Remarque : ce fichier utilise encore llama-3.3-70b-versatile, un modèle qui a été retiré du catalogue Groq depuis (constaté lors des travaux sur les projets 1, 4, 5 et 7, tous corrigés pour utiliser openai/gpt-oss-120b à la place). Ce fichier n'a pas été modifié à ce jour et échouera probablement avec une erreur model_not_found s'il est exécuté tel quel.

speech_to_text.py

La classe AudioAgent : transcription, appel IA, génération de PDF.

Lignes 1-12

```
import os
from groq import Groq
from fpdf import FPDF
from dotenv import load_dotenv
import arabic_reshaper
from bidi.algorithm import get_display

class AudioAgent:
    def __init__(self):
        """Initialise l'agent avec la clé API Groq"""
        load_dotenv()
        self.client = Groq(api_key=os.getenv("GROQ_API_KEY"))
```

Contrairement aux autres projets qui passent par le framework **Agno**, celui-ci appelle directement le **client officiel Groq** (from groq import Groq). La classe AudioAgent encapsule toute la logique métier ; son constructeur charge la clé API et crée le client.

Lignes 14-22

```
def transcribe_audio(self, audio_file_path, language="en"):
    """Transcrit un fichier audio avec Groq Whisper"""
    with open(audio_file_path, "rb") as audio_file:
        transcription = self.client.audio.transcriptions.create(
            file=("audio.mp3", audio_file.read()),
            model="whisper-large-v3",
            language=language,
        )
    return transcription
```

Envoie le fichier audio (lu en binaire) à l'API Whisper de Groq via client.audio.transcriptions.create(). Le paramètre language ("en" ou "ar" — Whisper utilise le code "ar" pour tous les dialectes arabes, y compris la darija) améliore la précision de la transcription.

Lignes 24-46

```
def process_with_gpt(self, transcript_text, choice, language="en"):
    """Traite le texte avec Groq LLaMA selon le choix (résumé ou explication)"""
    if language == "darija":
        prompt = (... instructions en darija, reponse exigee en arabe standard ...)
        system_msg = "(...texte arabe...) Répondez en arabe standard."
    elif language == "ar":
        prompt = (... texte en arabe standard ...)
        system_msg = "(...texte arabe...)"
    else:
        prompt = (
            f"Summarize the following transcript:\n{transcript_text}"
            if choice == "Summarize"
            else f"Explain in detail the following transcript:\n{transcript_text}"
        )
        system_msg = "You are an expert transcription assistant."
```

Construit un prompt et un message système différents selon 3 cas de figure : darija (instructions en darija, réponse exigée en arabe standard), arabe standard, ou anglais. Dans chaque cas, le choix de l'utilisateur (Summarize / Explain) détermine si le prompt demande un résumé ou une explication détaillée. C'est une approche *manuelle* de gestion multilingue (trois blocs de texte écrits en dur), à comparer avec l'approche plus générique des projets 1 et 5 (une seule instruction demandant au modèle de détecter la langue lui-même).

Lignes 48-55

```

response = self.client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {"role": "system", "content": system_msg},
        {"role": "user", "content": prompt}
    ]
)
return response.choices[0].message.content.strip()

```

Appel classique à l'API de complétion de chat de Groq (format compatible OpenAI), avec un message système et un message utilisateur. Retourne le texte de la réponse, débarrassé des espaces superflus.

Lignes 57-80

```

def _create_pdf(self, text, output_path, is_arabic=False):
    """Crée un PDF avec support Unicode et arabe (RTL)"""
    pdf = FPDF()
    pdf.add_page()
    pdf.set_auto_page_break(auto=True, margin=15)

    font_path = "C:/Windows/Fonts/arial.ttf"
    if os.path.exists(font_path):
        pdf.add_font("ArialUni", "", font_path, uni=True)
        pdf.set_font("ArialUni", size=14 if is_arabic else 12)
    else:
        pdf.set_font("Arial", size=12)

    if is_arabic:
        reshaped_text = arabic_reshaper.reshape(text)
        display_text = get_display(reshaped_text)
        pdf.multi_cell(0, 10, display_text, align="R")
    else:
        pdf.multi_cell(0, 10, text)

    pdf.output(output_path)
    return output_path

```

Génère un PDF à partir d'un texte. Charge la police Arial de Windows en mode Unicode (uni=True) si elle est disponible. Pour l'arabe, applique le même traitement que dans le projet 1 (arabic_reshaper + get_display) et aligne le texte à droite. Contrairement à export_chat.py du projet 1, le PDF est écrit directement sur disque (pdf.output(output_path)) plutôt que gardé en mémoire.

Lignes 82-88

```

def create_transcript_pdf(self, transcript_text, is_arabic=False):
    """Crée un PDF de la transcription"""
    return self._create_pdf(transcript_text, "transcript.pdf", is_arabic)

def create_gpt_pdf(self, gpt_output, is_arabic=False):
    """Crée un PDF de la sortie AI"""
    return self._create_pdf(gpt_output, "gpt_output.pdf", is_arabic)

```

Deux fonctions raccourcies qui appellent _create_pdf() avec des noms de fichiers fixes : transcript.pdf pour la transcription brute, gpt_output.pdf pour la réponse IA.

L'interface Streamlit : sélection de langue, upload audio, actions IA.

Lignes 1-14

```
import streamlit as st
from dotenv import load_dotenv
import os
from speech_to_text import AudioAgent

load_dotenv()
agent = AudioAgent()

st.title(" Speech to Text + AI Summary/Explanation")
st.caption("Powered by Groq (Whisper + LLaMA)")
```

Crée une instance unique de AudioAgent au chargement du script. Contrairement au projet 1, cette instance n'est pas mise en cache via `@st.cache_resource` : elle est recrée à chaque rechargement complet de la page (impact mineur ici car AudioAgent ne charge aucun modèle lourd localement, juste un client API léger).

Lignes 16-32

```
lang = st.selectbox(
    "Choose audio language / [texte arabe]",
    ["English", "[arabe] (Arabic)", "[arabe] (Darija)"]
)
if "Darija" in lang:
    lang_code = "darija"
    whisper_lang = "ar"
    is_arabic = True
elif "Arabic" in lang:
    lang_code = "ar"
    whisper_lang = "ar"
    is_arabic = True
else:
    lang_code = "en"
    whisper_lang = "en"
    is_arabic = False
```

Menu déroulant à trois choix. Le code dérive ensuite trois variables à partir du choix : `lang_code` (utilisé pour choisir le bon prompt dans `process_with_gpt`), `whisper_lang` (le code langue à 2 lettres attendu par Whisper — remarquez que darija et arabe standard partagent tous deux "ar"), et `is_arabic` (booléen utilisé partout ailleurs dans le fichier pour basculer les textes de l'interface entre anglais et arabe).

Lignes 34-52

```
uploaded_audio = st.file_uploader("Upload your audio file (.mp3, .wav)", type=["mp3", "wav"])
if uploaded_audio is not None:
    with open("temp_audio_file", "wb") as f:
        f.write(uploaded_audio.getbuffer())

    spinner_text = "[texte arabe]" if is_arabic else "Transcribing with Whisper..."
    with st.spinner(spinner_text):
        try:
            transcript = agent.transcribe_audio("temp_audio_file", language=whisper_lang)
            transcript_text = transcript.text
        except Exception as e:
            transcript_text = None
            st.error(f"Transcription failed: {e}")
```

Le fichier audio uploadé est d'abord écrit sur disque sous un nom fixe `temp_audio_file` (**ce fichier n'est jamais nettoyé après usage, et un second utilisateur simultanément écraserait le fichier du premier**), car `transcribe_audio()` attend un chemin de fichier. Le spinner et les messages qui suivent basculent tous entre arabe et anglais selon `is_arabic`.

Lignes 50-79

```
if transcript_text:
    st.success(...)
    st.text_area("...", transcript_text, height=200)

    btn_label = "[texte arabe]" if is_arabic else "Download transcript PDF"
    if st.button(btn_label):
        pdf_path = agent.create_transcript_pdf(transcript_text, is_arabic=is_arabic)
        with open(pdf_path, "rb") as f:
            st.download_button(..., f, file_name="transcript.pdf")

    options = ["Summarize", "Explain in detail"]
    choice = st.radio("radio_label", options)

    run_label = "[texte arabe]" if is_arabic else "Run AI"
    if st.button(run_label):
        with st.spinner(spinner_ai):
            ai_output = agent.process_with_gpt(transcript_text, choice, language=lang_code)
            st.text_area(..., ai_output, height=300)

    btn_ai_label = "[texte arabe]" if is_arabic else "Download AI Output PDF"
    if st.button(btn_ai_label):
        pdf_path = agent.create_gpt_pdf(ai_output, is_arabic=is_arabic)
        with open(pdf_path, "rb") as f:
            st.download_button(..., f, file_name="ai_output.pdf")
```

Affiche la transcription dans une zone de texte, avec un bouton d'export PDF qui génère le fichier (`create_transcript_pdf`) puis le relit immédiatement du disque pour le proposer au téléchargement via `st.download_button`. L'utilisateur choisit ensuite entre résumé et explication (`st.radio`), lance l'IA, puis peut de la même façon exporter la réponse en PDF. **Remarque structurelle : les boutons « Download PDF » sont imbriqués à l'intérieur du bloc `if st.button(...)` qui les précède** — dans Streamlit, cela signifie qu'ils disparaissent au clic suivant jusqu'à ce que le bouton parent soit à nouveau cliqué, un comportement parfois surprenant pour l'utilisateur.

Projet 3 — 3-Graphiti

Suivi d'activités sportives dans un graphe (Neo4j), avec une version simulée hors-ligne

Objectif

Démontrer comment modéliser des données de suivi sportif (courses, vélo, musculation...) dans une **base de données en graphe Neo4j**, puis interroger ces données avec des requêtes comme « les activités récentes d'un utilisateur » ou « les courses de plus de 5 km des 7 derniers jours ». Le dossier fournit **deux versions identiques en surface** : `grahp_caracts_customers.py` se connecte à un vrai serveur Neo4j (nécessite des identifiants), tandis que `demo_offline.py` simule le même comportement entièrement en mémoire, sans aucune dépendance externe — utile pour tester la logique sans installer Neo4j. Aucun modèle d'IA n'est utilisé dans ce projet.

Outils utilisés

Outil	Rôle dans ce projet
neo4j (driver Python officiel)	Se connecte à une base de données en graphe Neo4j et exécute des requêtes Cypher de manière asynchrone (uniquement dans la version « réelle »).
Aucun (version hors-ligne)	<code>demo_offline.py</code> remplace Neo4j par une simple classe Python stockant les données dans un dictionnaire en mémoire.
asyncio	Les deux versions sont écrites en style asynchrone (<code>async/await</code>), même la version hors-ligne, pour garder une interface identique entre les deux.

demo_offline.py

Version simulée (sans Neo4j) : même logique, stockage en mémoire.

Lignes 1-29

```
import asyncio
import json
import logging
import os
import sys
from datetime import datetime, timedelta, timezone

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger(__name__)

def iso_timestamp_days_ago(days: int) -> str:
    """Return ISO timestamp N days ago"""
    return (datetime.now(timezone.utc) - timedelta(days=days)).isoformat()
```

Force l'encodage UTF-8 en sortie sur Windows (évite les erreurs d'affichage d'emojis dans la console, un problème rencontré également dans le projet 7). Configure un logger standard. `iso_timestamp_days_ago()` est une fonction utilitaire qui calcule une date/heure passée (aujourd'hui moins N jours) au format ISO 8601, utilisée pour filtrer les activités « récentes ».

Lignes 32-66

```

class InMemoryStore:
    """Simulates Neo4j graph database operations in-memory"""

    def __init__(self):
        self.activities = {} # activity_id -> activity_data
        self.constraints = set()
        self.indices = set()

    async def create_constraint(self, name):
        self.constraints.add(name)

    async def create_index(self, name):
        self.indices.add(name)

    async def merge_activity(self, activity_data):
        self.activities[activity_data["activity_id"]] = {
            **activity_data,
            "created_at": datetime.now(timezone.utc).isoformat(),
        }

    async def query(self, user=None, activity_type=None, min_distance=None, since_time=None):
        results = []
        for act in self.activities.values():
            if user and act["user"] != user:
                continue
            if activity_type and act["activity_type"] != activity_type:
                continue
            if min_distance is not None and act["distance_km"] < min_distance:
                continue
            if since_time and act["timestamp"] < since_time:
                continue
            results.append(act)
        results.sort(key=lambda x: x["timestamp"], reverse=True)
        return results

```

Le cœur de la simulation : une classe qui imite l'API d'un pilote Neo4j mais stocke tout dans un simple dictionnaire Python (self.activities). merge_activity() imite l'opération Cypher MERGE (insère ou met à jour une activité par son identifiant). query() filtre manuellement en Python (boucle for avec plusieurs continue) ce qu'une vraie requête Cypher ferait côté serveur : par utilisateur, type d'activité, distance minimale et ancienneté, puis trie les résultats du plus récent au plus ancien.

Lignes 69-93

```

class FitnessTracker:
    def __init__(self):
        self.store = None

    async def connect(self):
        """Initialize in-memory store (simulates Neo4j connection)"""
        try:
            self.store = InMemoryStore()
            await self.store.create_constraint("activity_id_unique")
            await self.store.create_index("user_index")
            await self.store.create_index("activity_type_index")
            await self.store.create_index("timestamp_index")
            logger.info(" Connected to In-Memory Store (Neo4j simulation)")
        except Exception as e:
            logger.error(f" Failed to initialize store: {e}")
            raise

    async def close(self):
        """Close connection"""
        if self.store:
            self.store = None
            logger.info(" Connection closed")

```

La classe publique utilisée par le reste du script. connect() crée le magasin en mémoire et « déclare » quatre contraintes/index (qui n'ont aucun effet réel ici, mais reflètent fidèlement les appels qui seraient faits sur un vrai

Neo4j). close() libère simplement la référence.

Lignes 95-138

```
async def add_activity(self, activity_data):
    """Add a fitness activity"""
    await self.store.merge_activity(activity_data)
    logger.info(f" Added activity: {activity_data['activity_type']} - {activity_data['distance_km']}km")

async def query_recent_activities(self, user: str, days: int):
    """Query activities in the last N days"""
    since_time = iso_timestamp_days_ago(days)
    return await self.store.query(user=user, since_time=since_time)

async def query_running_over_distance(self, user: str, min_distance: float, days: int):
    """Query running activities over certain distance"""
    since_time = iso_timestamp_days_ago(days)
    return await self.store.query(
        user=user, activity_type="running", min_distance=min_distance, since_time=since_time,
    )

async def query_by_activity_type(self, user: str, activity_type: str, days: int):
    """Query activities of specific type"""
    since_time = iso_timestamp_days_ago(days)
    return await self.store.query(user=user, activity_type=activity_type,
    since_time=since_time)
```

Quatre méthodes métier (avec gestion d'erreur try/except omise ici par souci de place, présente dans le vrai fichier). add_activity() enregistre une activité. Les trois méthodes query_* sont de simples façades qui calculent la date de départ (since_time) puis délèguent à store.query() avec des filtres différents — respectivement : toutes les activités récentes d'un utilisateur, ses courses au-delà d'une distance minimale, ou ses activités d'un type précis.

Lignes 141-184

```
def create_sample_activities():
    """Create sample fitness activities"""
    return [
        {
            "user": "ahmed_hassan", "activity_id": "activity_a1", "activity_type": "running",
            "distance_km": 6.0, "duration_min": 35, "timestamp": iso_timestamp_days_ago(7),
        },
        {
            "user": "fatima_ali", "activity_id": "activity_a2", "activity_type": "cycling",
            "distance_km": 20.0, "duration_min": 60, "timestamp": iso_timestamp_days_ago(3),
        },
        # ... 3 autres activités pour omar_mahmoud et à nouveau ahmed_hassan/fatima_ali ...
    ]
```

Jeu de données de démonstration : 5 activités réparties entre 3 utilisateurs fictifs (ahmed_hassan, fatima_ali, omar_mahmoud), avec des types variés (course, vélo, musculation) et des dates étalées sur les 7 derniers jours, conçues pour bien illustrer les différents filtres des requêtes.

Lignes 187-251

```

def print_results(results, title: str):
    """Print query results"""
    print(f"\n--- {title} ---")
    if not results:
        print("No results found.")
        return
    for activity in results:
        print(f" • User: {activity['user']}")
        print(f" Activity: {activity['activity_type']}")
        # ...

async def main():
    print("Neo4j Fitness Tracker – OFFLINE DEMO MODE")
    tracker = FitnessTracker()
    try:
        await tracker.connect()
        activities = create_sample_activities()
        for activity in activities:
            await tracker.add_activity(activity)

        recent_results = await tracker.query_recent_activities("ahmed_hassan", days=5)
        print_results(recent_results, "Ahmed's Activities in the last 5 days")

        running_results = await tracker.query_running_over_distance("ahmed_hassan",
min_distance=5, days=7)
        print_results(running_results, "Ahmed's Running > 5 km in the last 7 days")
        # ... autres requetes de demonstration ...
    finally:
        await tracker.close()

```

`print_results()` affiche joliment une liste d'activités dans la console. `main()` est le scénario de démonstration complet : connexion, insertion des 5 activités exemples, puis enchainement de plusieurs requêtes (activités récentes, courses longues, activités par type, puis un tour de tous les utilisateurs) dont les résultats sont affichés au fur et à mesure. Le `finally: await tracker.close()` garantit la fermeture même en cas d'erreur.

Lignes 254-261

```

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print("\n Goodbye!")
    except Exception as e:
        print(f" Error: {e}")

```

Point d'entrée du script : lance la boucle asynchrone `main()`, avec une gestion propre de l'interruption clavier (Ctrl+C) et des erreurs imprévues.

grahp_caracts_custumers.py

Version réelle : même scénario, mais sur un vrai serveur Neo4j via des requêtes Cypher.

Ce fichier reproduit exactement le même scénario que `demo_offline.py` (mêmes données d'exemple, mêmes requêtes, même déroulé dans `main()`), seule la couche de stockage change : au lieu d'un dictionnaire Python, chaque opération est traduite en requête **Cypher** (le langage de requête de Neo4j) exécutée sur un vrai serveur. Seules les parties qui diffèrent sont détaillées ci-dessous.

Lignes 21-29

```
load_dotenv()

NEO4J_URI = os.environ.get("NEO4J_URI", "bolt://localhost:7687")
NEO4J_USER = os.environ.get("NEO4J_USER", "neo4j")
NEO4J_PASSWORD = os.environ.get("NEO4J_PASSWORD", None)

if not NEO4J_URI or not NEO4J_USER or not NEO4J_PASSWORD:
    raise ValueError(" You must set NEO4J_URI, NEO4J_USER, and NEO4J_PASSWORD in your .env file")
```

Contrairement à la version hors-ligne, ce fichier exige trois variables d'environnement pour se connecter à un vrai serveur Neo4j (URI de connexion au format `bolt://`, utilisateur, mot de passe) et s'arrête immédiatement avec une erreur explicite si l'une d'elles manque.

Lignes 39-58

```
async def connect(self):
    """Initialize connection to Neo4j"""
    self.driver = AsyncGraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))
    async with self.driver.session() as session:
        await session.run("RETURN 1")

    async with self.driver.session() as session:
        await session.run("CREATE CONSTRAINT activity_id IF NOT EXISTS FOR (a:Activity) REQUIRE a.activity_id IS UNIQUE")
        await session.run("CREATE INDEX user_index IF NOT EXISTS FOR (a:Activity) ON (a.user)")
        await session.run("CREATE INDEX activity_type_index IF NOT EXISTS FOR (a:Activity) ON (a.activity_type)")
        await session.run("CREATE INDEX timestamp_index IF NOT EXISTS FOR (a:Activity) ON (a.timestamp)")
```

Crée un vrai pilote de connexion asynchrone Neo4j (`AsyncGraphDatabase.driver`), teste la connexion avec une requête triviale (`RETURN 1`), puis crée réellement une contrainte d'unicité sur `activity_id` et trois index Neo4j (sur l'utilisateur, le type d'activité et l'horodatage) — exactement les opérations que la version hors-ligne se contentait de simuler.

Lignes 66-84

```
async def add_activity(self, activity_data):
    """Add a fitness activity to Neo4j"""
    async with self.driver.session() as session:
        query = """
        MERGE (a:Activity {activity_id: $activity_id})
        SET a.user = $user,
            a.activity_type = $activity_type,
            a.distance_km = $distance_km,
            a.duration_min = $duration_min,
            a.timestamp = $timestamp,
            a.created_at = datetime()
        """
        await session.run(query, activity_data)
```

L'équivalent Cypher de `merge_activity()` : `MERGE` crée le nœud `Activity` s'il n'existe pas encore (identifié par `activity_id`), puis `SET` renseigne/actualise tous ses attributs. Les valeurs préfixées par `$` sont des paramètres liés

(passés via `activity_data`), une pratique sûre qui évite les injections Cypher.

Lignes 86-147

```
async def query_recent_activities(self, user: str, days: int):
    async with self.driver.session() as session:
        query = """
            MATCH (a:Activity)
            WHERE a.user = $user AND a.timestamp >= $since_time
            RETURN a.user as user, a.activity_type as activity_type,
                   a.distance_km as distance_km, a.duration_min as duration_min, a.timestamp as
timestamp
            ORDER BY a.timestamp DESC
            """
        result = await session.run(query, {"user": user, "since_time": since_time})
        activities = []
        async for record in result:
            activities.append({...})
        return activities

# query_running_over_distance() et query_by_activity_type() suivent exactement le même
# schéma MATCH/WHERE/RETURN, avec une condition WHERE supplémentaire à chaque fois.
```

Les trois méthodes de requête suivent toutes le même schéma Cypher : `MATCH (a:Activity)` sélectionne tous les nœuds Activity, `WHERE` applique les filtres (utilisateur, type, distance minimale, ancienneté — exactement les mêmes critères que la version Python de `InMemoryStore.query()`, mais exécutés côté serveur Neo4j), et `ORDER BY a.timestamp DESC` trie du plus récent au plus ancien. Le résultat est un curseur asynchrone (`async for record in result`) qu'on transforme en liste de dictionnaires Python.

Les fonctions `create_sample_activities()`, `print_results()` et `main()` sont, ligne pour ligne, identiques à celles de `demo_offline.py` — c'est justement ce qui permet de comparer directement les deux approches (simulation vs. base réelle) sans changer la logique métier.

Projet 4 — 4-FireCrawl_PDFparsing

Analyse intelligente d'un PDF en ligne : extraction + questions/réponses

Objectif

Donner à l'utilisateur l'URL d'un PDF public (rapport, étude...) et lui permettre de discuter avec son contenu comme dans un chat, dans n'importe quelle langue. Le PDF est récupéré et converti en texte par le service **Firecrawl**, mis en cache pour éviter de consommer des crédits à chaque question, puis chaque question est répondue par un agent IA qui cite ses sources. La conversation peut être exportée en PDF (avec prise en charge de l'arabe).

Outils utilisés

Outil	Rôle dans ce projet
Streamlit	Interface : champ URL, bouton de réanalyse, fil de discussion.
Firecrawl	Service qui télécharge un PDF distant et le convertit en texte/markdown propre.
Groq (Agno, gpt-oss-120b)	Modèle qui répond aux questions en se basant sur le contenu extrait.
fpdf2 + arabic-reshape r/python-bidi	Génèrent le PDF d'export de la conversation, avec prise en charge de l'arabe.

pdf_analyzer.py

Extraction du PDF via Firecrawl + agent de questions/réponses.

Lignes 1-17

```
import re
import os
from firecrawl import FirecrawlApp
from agno.agent import Agent
from agno.models.groq import Groq
from textwrap import dedent
from dotenv import load_dotenv

load_dotenv()
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
FIRECRAWL_API_KEY = os.getenv("FIRECRAWL_API_KEY")

# gpt-oss-120b supporte une fenêtre de contexte large (~131k tokens), mais le
# compte gratuit Groq est plafonné à 8000 tokens/minute (TPM) en on-demand :
# on reste nettement en dessous pour ne pas se faire rejeter par ce quota.
MAX_CONTEXT_CHARS = 20000
```

Charge les deux clés API nécessaires. MAX_CONTEXT_CHARS plafonne la quantité de texte du PDF envoyée au modèle à chaque question : une valeur découverte expérimentalement pour rester sous le quota gratuit Groq de 8000 tokens par minute (une première tentative à 60 000 caractères avait échoué avec une erreur `rate_limit_exceeded`).

Lignes 19-44

```
firecrawl = FirecrawlApp(api_key=FIRECRAWL_API_KEY)

pdf_agent = Agent(
    model=Groq(api_key=GROQ_API_KEY, id="openai/gpt-oss-120b"),
    name="PDF Analysis Agent",
    role="Expert en analyse de documents PDF",
    instructions=dedent("""
    Vous êtes un expert en analyse de documents PDF. Votre rôle est :
    1. Répondre aux questions précises sur le document
    2. Identifier les thèmes clés et structure logique
    ...
    - Répondre toujours dans la même langue que la question posée (français, arabe ou anglais)

    Format de réponse :
    Réponse concise
    Contexte : [explication ou justification]
    Source : [citation ou référence précise]
    """),
    markdown=True
)
```

Initialise le client Firecrawl, puis définit l'agent Agno chargé de répondre aux questions. Les instructions imposent un format de réponse structuré en trois parties (réponse, contexte, source) et exigent que l'agent réponde dans la langue de la question.

Lignes 46-87

```

def extract_pdf_content(pdf_url):
    try:
        try:
            result = firecrawl.scrape(url=pdf_url, formats=['markdown'])
        except AttributeError:
            try:
                result = firecrawl.scrape_url(url=pdf_url)
            except AttributeError:
                raise Exception("Méthode Firecrawl non trouvée...")

        markdown_text = None
        if hasattr(result, 'markdown'):
            markdown_text = result.markdown
        elif hasattr(result, "data") and isinstance(result.data, list) and len(result.data) > 0:
            markdown_text = getattr(result.data[0], "markdown", None)
        elif isinstance(result, dict) and 'markdown' in result:
            markdown_text = result['markdown']

        if not markdown_text:
            return None, "Aucune donnée extraite. Vérifiez que le PDF n'est pas protégé."

        cleaned = re.sub(r'!\[.*?\]\(.*?\)', '', markdown_text) # supprime les images
        cleaned = re.sub(r'#+.*', '', cleaned, flags=re.MULTILINE) # supprime les titres
        cleaned = re.sub(r'\n{3,}', '\n\n', cleaned) # lignes vides multiples
        extracted_text = cleaned.strip()

        if not extracted_text:
            return None, "Le contenu extrait est vide après nettoyage."
        return extracted_text, None
    except Exception as e:
        return None, f"Erreur lors de l'extraction: {e}"

```

Extrait le texte du PDF. Le bloc try/except AttributeError imbriqué gère deux versions différentes possibles de la bibliothèque Firecrawl (scrape() en v2, scrape_url() en v1). Le résultat peut aussi avoir plusieurs formes selon la version (attribut .markdown, liste .data, ou dictionnaire), d'où les trois vérifications successives. Enfin, trois expressions régulières nettoient le markdown : suppression des images, des titres (lignes commençant par #), et compression des lignes vides multiples.

Lignes 89-110

```

def analyze_question(question, pdf_content, history=None, max_retries=2):
    history_block = ""
    if history:
        recent = history[-6:]
        turns = [f"{'Utilisateur' if h['role'] == 'user' else 'Assistant'} : {h['content']}" for h
in recent]
        history_block = "\n\nHistorique récent de la conversation :\n" + "\n".join(turns)

    content_limit = MAX_CONTEXT_CHARS
    last_error = None
    for _ in range(max_retries + 1):
        prompt = f"{question}{history_block}\n\nContenu PDF extrait
:\n{pdf_content[:content_limit]}"
        try:
            answer = pdf_agent.run(prompt)
            return answer.content, None
        except Exception as e:
            last_error = e
            if "rate_limit" in str(e).lower() or "tokens" in str(e).lower():
                content_limit = content_limit // 2
    return None, f"Erreur lors de l'analyse (après {max_retries + 1} tentatives) : {last_error}"

```

Construit le prompt en incluant les 3 derniers échanges de la conversation (pour le contexte de suivi) et un extrait du PDF. La boucle de nouvel essai gère **deux** types d'erreurs différemment : si le message d'erreur contient « rate_limit » ou « tokens » (quota dépassé), elle **divise par deux** la quantité de texte envoyée avant de réessayer ; pour toute autre erreur (comme un appel d'outil transitoire mal formé côté Groq), elle réessaie simplement à l'identique.

Interface : URL du PDF, mise en cache, chat de questions/réponses.

Lignes 1-22

```
import streamlit as st
from export_pdf import build_pdf
from pdf_analyzer import extract_pdf_content, analyze_question

st.set_page_config(page_title="Analyse PDF IA", page_icon="", layout="wide")
st.title(" Analyse Intelligente de PDF")

DEFAULT_URL =
"https://www.pwc.com/gx/en/issues/analytics/assets/pwc-ai-analysis-sizing-the-prize-report.pdf"

if "pdf_url_cached" not in st.session_state:
    st.session_state.pdf_url_cached = None
    st.session_state.pdf_text = None
    st.session_state.messages = []

col_url, col_btn = st.columns([5, 1])
with col_url:
    pdf_url = st.text_input(" URL du PDF :", value=DEFAULT_URL)
with col_btn:
    force_refresh = st.button(" Réanalyser", use_container_width=True)
```

Initialise trois variables de session : l'URL déjà analysée (pour détecter un changement), le texte extrait, et l'historique de chat. Deux éléments côte à côte : le champ URL et un bouton « Réanalyser » pour forcer une nouvelle extraction même si l'URL n'a pas changé.

Lignes 24-39

```
if pdf_url and (pdf_url != st.session_state.pdf_url_cached or force_refresh):
    with st.spinner("■ Extraction du contenu en cours..."):
        extracted_text, error = extract_pdf_content(pdf_url)
        st.session_state.pdf_url_cached = pdf_url
    if error:
        st.error(f" {error}")
        st.session_state.pdf_text = None
    else:
        st.session_state.pdf_text = extracted_text
        st.session_state.messages = []
        st.success(" Contenu extrait avec succès.")
elif pdf_url == st.session_state.pdf_url_cached and st.session_state.pdf_text is None:
    st.error(" La dernière extraction a échoué. Modifie l'URL ou clique sur Réanalyser.")
```

La ligne clé de ce fichier : Firecrawl n'est appelé **que si l'URL a changé ou que le bouton Réanalyser a été cliqué**. Sans ce garde-fou, poser une simple question relancerait une extraction complète (et coûteuse en crédits Firecrawl) à chaque fois, puisque Streamlit ré-exécute tout le script à chaque interaction. Le elif gère le cas où la dernière tentative a échoué, pour afficher l'erreur sans redemander une extraction en boucle.

Lignes 41-58

```

if st.session_state.pdf_text:
    with st.expander(" Voir le contenu extrait (nettoyé)"):
        st.text_area("Contenu PDF", st.session_state.pdf_text, height=400)

    st.subheader(" Posez vos questions au sujet du PDF")

    for msg in st.session_state.messages:
        with st.chat_message(msg["role"]):
            st.markdown(msg["content"])

    if st.session_state.messages:
        st.download_button(
            " Exporter la conversation en PDF",
            data=build_pdf(st.session_state.pdf_url_cached, st.session_state.messages),
            file_name="analyse_pdf_conversation.pdf",
            mime="application/pdf",
        )

```

Affiche le contenu extrait dans un panneau repliable, réaffiche l'historique de conversation, puis (si des messages existent) propose le bouton d'export PDF, qui appelle `build_pdf()` du fichier `export_pdf.py` (structure identique à `export_chat.py` du projet 1).

Lignes 60-76

```

question = st.chat_input("Ex: Quel est l'impact économique de l'IA ?")
if question:
    st.session_state.messages.append({"role": "user", "content": question})
    with st.chat_message("user"):
        st.markdown(question)

    with st.chat_message("assistant"):
        with st.spinner(" Analyse de la question par l'agent..."):
            answer, error = analyze_question(
                question, st.session_state.pdf_text, history=st.session_state.messages[:-1]
            )
        if error:
            answer = f" {error}"
        st.markdown(answer)
    st.session_state.messages.append({"role": "assistant", "content": answer})
    st.rerun()

```

Capture la question, l'ajoute à l'historique, puis appelle `analyze_question()` en lui passant tout l'historique **sauf** la question qu'on vient d'ajouter (`messages[:-1]`, pour éviter de la dupliquer dans le prompt). La réponse (ou le message d'erreur) est affichée puis ajoutée à son tour à l'historique, et `st.rerun()` rafraîchit la page.

export_pdf.py

Génère le PDF d'export de la conversation — structure jumelle de export_chat.py (projet 1).

Ce fichier est quasi identique à la partie PDF de export_chat.py du projet 1 (mêmes fonctions contains_arabic(), _shape_line(), _register_pdf_font(), _write_line()). Seule build_pdf() diffère légèrement : elle inclut l'URL du document source au lieu de la liste des articles, et libelle les messages « Question » / « Réponse » plutôt que « Vous » / « Assistant ».

Lignes 53-77

```
def build_pdf(pdf_url: str, messages: list[dict]) -> bytes:
    pdf = FPDF()
    pdf.add_page()
    font = _register_pdf_font(pdf)

    pdf.set_font(font, "B", 16)
    _write_line(pdf, 10, "Analyse PDF IA - Export de la conversation", align="C")
    pdf.ln(2)

    pdf.set_font(font, "", 9)
    _write_line(pdf, 6, f"Exporté le {datetime.now().strftime('%d/%m/%Y %H:%M')}")
    _write_line(pdf, 6, f"Document source : {pdf_url}")
    pdf.ln(4)

    for msg in messages:
        label = "Question" if msg["role"] == "user" else "Réponse"
        pdf.set_font(font, "B", 11)
        _write_line(pdf, 7, _shape_line(f"{label} :"))
        pdf.set_font(font, "", 11)
        for line in msg["content"].split("\n"):
            _write_line(pdf, 7, _shape_line(line))
        pdf.ln(3)

    return bytes(pdf.output())
```

Titre centré, date d'export et URL source, puis chaque message étiqueté « Question » ou « Réponse ». Comme dans le projet 1, chaque ligne passe par _shape_line() pour gérer l'arabe correctement, et pdf.output() retourne directement les octets du fichier, sans écriture sur disque.

Projet 5 — 5-FireCrawl_Scrape

Scraper universel d'annonces (immobilier, véhicules, emploi, produits...) avec analyse IA

Objectif

Extraire automatiquement toutes les annonces d'une page web (immobilier, véhicules, offres d'emploi, produits, services...), quel que soit le site ou la langue, puis analyser chacune avec l'IA pour en dégager les atouts et les risques ou signaux d'alerte (prix anormal, informations manquantes, incohérences). Contrairement à une approche classique par expressions régulières spécifiques à un site, l'extraction est entièrement pilotée par un agent IA à sortie structurée, ce qui la rend générique. Les résultats peuvent être filtrés, exportés en CSV, et envoyés vers Airtable.

Outils utilisés

Outil	Rôle dans ce projet
Streamlit	Interface : barre latérale (source, export), métriques, filtre, cartes d'annonces repliables.
Firecrawl	Récupère le contenu markdown de la page à scraper.
Groq + Agno (sortie structurée Pydantic)	Extrait les annonces sous forme d'objets validés (pas de regex), puis génère un résumé/analyse par annonce.
Pydantic	Définit le schéma exact (référence, titre, prix, localisation, description) que l'IA doit remplir.
langdetect	Détecte la langue de chaque annonce, pour afficher un drapeau et forcer la langue de réponse de l'IA.
requests	Envoie les annonces analysées vers l'API Airtable (optionnel).

scraping.py

Le moteur : extraction structurée par IA, analyse, détection de langue, export Airtable.

Lignes 1-20

```
import os
from textwrap import dedent
from typing import Optional

import requests
from dotenv import load_dotenv
from firecrawl import Firecrawl
from langdetect import DetectorFactory, LangDetectException, detect
from pydantic import BaseModel, Field

from agno.agent import Agent
from agno.models.groq import Groq

DetectorFactory.seed = 0 # résultats de détection de langue déterministes

LANGUAGE_NAMES = {
    "fr": "French", "en": "English", "ar": "Arabic", ...
}
```

DetectorFactory.seed = 0 rend langdetect déterministe (sans cette ligne, la bibliothèque peut donner des résultats légèrement différents d'une exécution à l'autre sur un texte ambigu). LANGUAGE_NAMES fait correspondre les codes ISO (« fr », « en »...) à des noms anglais complets, utilisés plus loin pour instruire explicitement le modèle sur la langue de réponse attendue.

Lignes 32-38

```
MAX_CHUNK_CHARS = 12000
MAX_CHUNKS = 6
```

Deux limites cruciales pour la robustesse : MAX_CHUNK_CHARS découpe la page en morceaux d'au plus 12 000 caractères avant de les envoyer au modèle (reste sous le quota Groq de 8000 tokens/minute), et MAX_CHUNKS plafonne à 6 le nombre de morceaux traités — des tests réels ont montré qu'une page pouvait faire plus de 140 000 caractères une fois tous les filtres/menus inclus, ce qui rendrait le traitement complet beaucoup trop long et risqué pour le quota.

Lignes 44-58

```
class Listing(BaseModel):
    reference: Optional[str] = Field(None, description="Référence/ID de l'annonce si présent sur la page")
    title: Optional[str] = Field(None, description="Titre ou catégorie de l'annonce (ex: Maison, Toyota Corolla 2019, ...)")
    price: Optional[str] = Field(None, description="Prix, salaire ou tarif affiché tel quel, avec sa devise d'origine")
    location: Optional[str] = Field(None, description="Localisation si pertinente ...")
    description: str = Field("", description="Description de l'annonce, dans sa langue d'origine")

class ListingExtraction(BaseModel):
    listings: list[Listing] = Field(description="Toutes les annonces/offres distinctes trouvées dans le texte...")
```

Deux modèles **Pydantic** qui définissent le format exact que l'IA doit produire. C'est la clé de la généricité du projet : au lieu d'écrire du code de découpage spécifique à un site (comme text.split("Ref :") dans une version antérieure), on décrit simplement la *forme* des données attendues, et le modèle se charge de les extraire quel que soit le site ou la langue.

Lignes 61-82

```

extraction_agent = Agent(
    model=Groq(api_key=GROQ_API_KEY, id="openai/gpt-oss-120b"),
    name="Listing Extractor",
    role="Expert en extraction de données d'annonces multilingue, toutes catégories",
    instructions=dedent("""
Vous recevez le contenu brut (markdown) d'une page web listant des annonces ou offres...
Identifiez CHAQUE annonce distincte et extrayez, sans traduire ni reformuler :
- référence/ID si présent
- titre ou catégorie de l'annonce
- prix, salaire ou tarif (avec sa devise d'origine)
- localisation si pertinente pour ce type d'annonce
- description
...
"""),
    output_schema=ListingExtraction,
)

```

L'agent d'extraction. Le paramètre `output_schema=ListingExtraction` est ce qui force Agno/Groq à retourner directement un objet Python validé (une instance de `ListingExtraction`) plutôt qu'un texte libre à parser soi-même — c'est la fonctionnalité de « sortie structurée » du modèle.

Lignes 84-119

```

listing_agent = Agent(
    model=Groq(api_key=GROQ_API_KEY, id="openai/gpt-oss-120b"),
    name="Listing Analyzer",
    role="Expert en analyse d'annonces, toutes catégories",
    instructions=dedent("""
Votre mission pour chaque annonce est :
- Générer un résumé clair et concis, adapté à sa catégorie.
- Identifier les atouts principaux...
- Identifier les risques ou signaux d'alerte potentiels...
Règles strictes :
- Adaptez votre analyse à la catégorie réelle de l'annonce (ne pas parler de "quartier" pour
une
    offre d'emploi...)
- Répondez toujours dans la langue du champ "description"...
...
"""),
    markdown=True,
)

```

Le second agent, chargé d'analyser (et non plus d'extraire) chaque annonce individuelle. Ses instructions insistent explicitement sur l'adaptation au type d'annonce — une contrainte ajoutée après la généralisation du projet, pour éviter que le modèle ne parle de « quartier » pour une offre d'emploi par exemple.

Lignes 122-171

```

def _chunks(text: str, size: int) -> list[str]:
    return [text[i : i + size] for i in range(0, len(text), size)]

def extract_listings(content: str, progress_callback=None) -> list[dict]:
    records = []
    seen = set()
    all_chunks = _chunks(content, MAX_CHUNK_CHARS)[:MAX_CHUNKS]
    total = len(all_chunks)

    for i, chunk in enumerate(all_chunks, 1):
        if progress_callback:
            progress_callback(i, total)
        try:
            result = extraction_agent.run(chunk)
            data = result.content
        except Exception as e:
            print(f"Debug - échec d'extraction sur un morceau de page : {e}")
            continue

        if not isinstance(data, ListingExtraction):
            print(f"Debug - réponse non structurée sur le morceau {i}, ignoré : {str(data)[:200]}")
            continue

        for item in data.listings:
            key = (item.reference or "", item.price or "", (item.description or "")[:80])
            if key in seen:
                continue
            seen.add(key)
            records.append({"ref": item.reference or f"annonce-{len(records) + 1}", ...})
    return records

```

Découpe le contenu en morceaux, puis appelle l'agent d'extraction sur chacun successivement, en signalant la progression via `progress_callback`. Deux niveaux de résilience : si l'appel au modèle échoue (exception), ou si sa réponse n'est **pas** une instance valide de `ListingExtraction` (cas rare où le modèle ne respecte pas le schéma demandé), le morceau est simplement ignoré plutôt que de faire planter toute l'extraction. La key (référence+prix+début de description) sert à dédupliquer les annonces qui apparaîtraient dans deux morceaux qui se chevauchent.

Lignes 174-193

```

def scrape_and_parse(url, progress_callback=None):
    try:
        scrape_result = firecrawl.scrape(url, formats=["markdown"])
        markdown_content = None
        if hasattr(scrape_result, "markdown"):
            markdown_content = scrape_result.markdown
        elif isinstance(scrape_result, dict) and "markdown" in scrape_result:
            markdown_content = scrape_result["markdown"]

        if not markdown_content:
            raise ValueError("Impossible d'extraire le contenu de la page (aucun markdown retourné).")

        print(f"Debug - contenu récupéré : {len(markdown_content)} caractères")
        return extract_listings(markdown_content, progress_callback=progress_callback)
    except Exception as e:
        print(f"Erreur dans scrape_and_parse: {e}")
        raise

```

La fonction appelée par l'interface : récupère la page via Firecrawl (format markdown uniquement, plus simple que le HTML brut utilisé dans une version antérieure), puis délègue à `extract_listings()`.

Lignes 196-233

```

def detect_language_code(text: str) -> Optional[str]:
    if not text or len(text.strip()) < 8:
        return None
    try:
        return detect(text)
    except LangDetectException:
        return None

def get_ai_summary(record):
    description = record.get("description", "") or ""
    detected = _detect_language_name(description) or _detect_language_name(f"{record.get('title',
    '')} {description}")
    lang_instruction = f'Respond entirely in {detected}.' if detected else "Respond in the same
    language as the description below."

    prompt = f"""Listing / classified ad ({lang_instruction}):

    reference: {record.get('ref')}
    title: {record.get('title')}
    price: {record.get('price')}
    location: {record.get('location')}
    description: {description}
    """

    summary = listing_agent.run(prompt)
    return summary.content

```

get_ai_summary() détecte explicitement la langue de la description (via langdetect) plutôt que de laisser le modèle deviner tout seul : une description très courte comme « 4 beds 2 baths 1,527 sq. ft. » s'est avérée trop ambiguë pour que le modèle infère fidèlement l'anglais, et faisait parfois basculer la réponse en français par défaut (à cause des libellés français employés ailleurs dans les prompts du projet). L'instruction de langue est alors injectée explicitement dans le prompt (lang_instruction).

Lignes 236-258

```

def save_to_airtable(record):
    if not (AIRTABLE_API_KEY and AIRTABLE_BASE_ID and AIRTABLE_TABLE_ID):
        return False, "Airtable non configuré : ajoute AIRTABLE_API_KEY, AIRTABLE_BASE_ID et
    AIRTABLE_TABLE_ID au fichier .env."

    enriched_record = {"ref": ..., "price": ..., "location": ..., "description": ..., "summary":
    ...}

    response = requests.post(
        f"https://api.airtable.com/v0/{AIRTABLE_BASE_ID}/{AIRTABLE_TABLE_ID}",
        headers={"Authorization": f"Bearer {AIRTABLE_API_KEY}", "Content-Type":
    "application/json"},
        json={"fields": enriched_record},
    )
    return response.status_code == 200, response.text

```

Envoie une annonce (et son résumé IA) vers une base Airtable via son API REST. Vérifie d'abord que les trois identifiants Airtable sont configurés et retourne un message d'erreur clair sinon, plutôt que de laisser requests.post() échouer avec une URL malformée.

Interface : barre latérale, filtre, cartes d'annonces avec drapeau de langue détectée.

Lignes 1-26

```
import csv
import io
import streamlit as st
from scraping import detect_language_code, get_ai_summary, save_to_airtable, scrape_and_parse

LANGUAGE_FLAGS = {"fr": "", "en": "", "ar": "", ...}

st.set_page_config(page_title="Scraper d'Annonces IA", page_icon="", layout="wide")
st.title(" Scraper d'Annonces avec Analyse IA")

if "records" not in st.session_state:
    st.session_state.records = []
if "summaries" not in st.session_state:
    st.session_state.summaries = {}
if "scraped_url" not in st.session_state:
    st.session_state.scraped_url = None
```

csv/io servent à générer l'export CSV en mémoire. LANGUAGE_FLAGS associe chaque code langue à un emoji de drapeau, affiché à côté de chaque annonce. Trois variables de session : la liste des annonces trouvées, le dictionnaire des résumés IA déjà générés (indexé par référence d'annonce), et l'URL actuellement chargée.

Lignes 36-70

```
with st.sidebar:
    st.header(" Source")
    url = st.text_input("URL à scraper",
value="https://www.century21.fr/annonces/achat-maison/v-bordeaux/", ...)
    launch = st.button(" Lancer l'extraction", use_container_width=True)

    if st.session_state.records:
        st.header(" Résultats")
        csv_buffer = io.StringIO()
        writer = csv.DictWriter(csv_buffer, fieldnames=["ref", "title", "price", "location",
"description", "summary"])
        writer.writeheader()
        for r in st.session_state.records:
            writer.writerow(**r, "summary": st.session_state.summaries.get(r["ref"], ""))
        st.download_button(" Exporter en CSV", data=csv_buffer.getvalue().encode("utf-8-sig"),
...)

    if st.button(" Réinitialiser", use_container_width=True):
        st.session_state.records = []
        st.session_state.summaries = {}
        st.session_state.scraped_url = None
        st.rerun()
```

La barre latérale regroupe la source (URL + bouton de lancement) et, une fois des résultats disponibles, les actions globales : export CSV (construit ligne par ligne avec csv.DictWriter, chaque ligne combinant les données de l'annonce et son résumé IA s'il existe) et réinitialisation complète. encode("utf-8-sig") ajoute un BOM UTF-8 pour qu'Excel affiche correctement les caractères accentués et arabes à l'ouverture du CSV.

Lignes 72-95

```

if launch and url:
    progress_bar = st.progress(0.0, text=" Récupération de la page...")

    def _on_progress(step, total):
        progress_bar.progress(step / total, text=f" Analyse du contenu ({step}/{total})...")

    try:
        st.session_state.records = scrape_and_parse(url, progress_callback=_on_progress)
        st.session_state.summaries = {}
        st.session_state.scraped_url = url
        progress_bar.empty()
        if st.session_state.records:
            st.success(f" {len(st.session_state.records)} annonce(s) trouvée(s) !")
        else:
            st.warning(" Aucune annonce trouvée sur cette page.")
    except Exception as e:
        progress_bar.empty()
        st.error(f" Erreur : {e}")

```

Au clic sur « Lancer l'extraction » : affiche une vraie barre de progression (pas juste un spinner), mise à jour à chaque morceau de page traité via la fonction `_on_progress` passée en callback à `scrape_and_parse()`. Réinitialise les résumés à chaque nouvelle extraction (ils correspondent à l'ancienne liste d'annonces).

Lignes 102-119

```

col1, col2, col3 = st.columns(3)
col1.metric(" Annonces trouvées", len(st.session_state.records))
col2.metric(" Résumés IA générés", len(st.session_state.summaries))
col3.metric(" Progression", f"{len(st.session_state.summaries)}/{len(st.session_state.records)}")

search = st.text_input(" Filtrer (prix, lieu, description...)", ...)

to_analyze = [r for r in st.session_state.records if r["ref"] not in st.session_state.summaries]
if to_analyze and st.button(f" Analyser les {len(to_analyze)} annonce(s) restante(s) avec l'IA"):
    batch_progress = st.progress(0.0)
    for i, record in enumerate(to_analyze, 1):
        batch_progress.progress(i / len(to_analyze), text=f"Analyse {i}/{len(to_analyze)}...")
        st.session_state.summaries[record["ref"]] = get_ai_summary(record)
    batch_progress.empty()
    st.rerun()

```

Trois indicateurs chiffrés, un champ de recherche, et un bouton pour analyser **en une fois** toutes les annonces qui n'ont pas encore de résumé IA (`to_analyze`), avec sa propre barre de progression — un complément pratique au bouton d'analyse individuelle présent dans chaque carte d'annonce.

Lignes 123-149

```

records = st.session_state.records
if search:
    needle = search.lower()
    records = [r for r in records if needle in r["price"].lower() or needle in
r["location"].lower()
                or needle in r["description"].lower() or needle in (r.get("title") or "").lower()]

for i, record in enumerate(records, 1):
    title_suffix = f" - {record['title']}" if record.get("title") else ""
    lang_code = detect_language_code(record.get("description", ""))
    flag = LANGUAGE_FLAGS.get(lang_code, "") if lang_code else ""

    with st.expander(f"{flag} Annonce {i} - {record['ref']}{title_suffix}"):
        has_location = record["location"] not in ("", "Non précisée")
        if has_location:
            badge_col1, badge_col2 = st.columns(2)
            badge_col1.markdown(f"### {record['price']}")
            badge_col2.markdown(f"### {record['location']}")
        else:
            st.markdown(f"### {record['price']}")
            st.markdown(f"*** Description : ** {record['description']}")

```

Le filtre de recherche fait une simple recherche de sous-chaîne insensible à la casse sur quatre champs à la fois. Pour chaque annonce affichée, la langue de sa description est détectée à la volée pour choisir le drapeau du titre du panneau. La colonne « localisation » n'est affichée que si elle est réellement renseignée (has_location) — utile depuis la généralisation à toutes les catégories d'annonces, où un produit n'a par exemple pas de localisation.

Lignes 153-179

```
if record["ref"] in st.session_state.summaries:
    st.markdown("### Résumé IA")
    st.markdown(st.session_state.summaries[record["ref"]])
elif st.button(" Analyser cette annonce avec l'IA", key=f"analyze_{record['ref']}"):
    with st.spinner(" Génération de l'analyse..."):
        st.session_state.summaries[record["ref"]] = get_ai_summary(record)
    st.rerun()

if record["ref"] in st.session_state.summaries and st.button(
    f" Enregistrer annonce {i} dans Airtable", key=f"airtable_{record['ref']}"
):
    record_with_summary = {**record, "summary": st.session_state.summaries[record["ref"]]}
    success, message = save_to_airtable(record_with_summary)
    if success:
        st.success(" Enregistré dans Airtable")
    else:
        st.error(f" {message}")
```

Le résumé IA n'est généré **qu'à la demande**, via un bouton explicite — un point important : le contenu d'un `st.expander` s'exécute même fermé, donc sans ce bouton, afficher 17 annonces déclencherait immédiatement 17 appels IA, même pour des annonces jamais consultées (bug corrigé après l'avoir observé en test). La clé unique de chaque bouton utilise la référence de l'annonce (`record['ref']`) plutôt que sa position dans la liste, pour rester stable même quand le filtre de recherche change l'ordre affiché. Le bouton Airtable n'apparaît qu'une fois le résumé disponible.

Projet 7 — 7-UQLM

Détecteur d'hallucinations : quantification d'incertitude avant d'accepter une réponse IA

Objectif

Illustrer une technique de fiabilisation des réponses d'un modèle de langage : au lieu de faire confiance aveuglément à une réponse, on génère **plusieurs réponses** à la même question et on mesure leur **incertitude** (à quel point elles sont cohérentes entre elles). Un agent IA décide ensuite, en se basant **uniquement** sur ce score d'incertitude (jamais sur la bonne réponse, qu'il ne connaît pas), s'il faut **ACCEPTER**, **REFORMULER** ou **REFUSER** la réponse produite. Le projet teste cette approche sur des questions mathématiques simples (jeu de données SVAMP), avec une interface permettant aussi de tester ses propres questions.

Outils utilisés

Outil	Rôle dans ce projet
uqlm (BlackBoxUQ)	Bibliothèque de quantification d'incertitude : génère plusieurs réponses par question et calcule des scores de cohérence entre elles.
langchain-groq	Fournit le modèle utilisé par uqlm pour générer les réponses candidates (interface LangChain standard).
Agno + Groq	L'agent qui prend la décision finale (ACCEPTER/REFORMULER/REFUSER) à partir du score d'incertitude.
sentence-transformers (interne à uqlm)	Calcule la similarité cosinus entre les réponses générées, un des quatre scores d'incertitude.
pandas	Manipule les résultats sous forme de tableau, exporté en CSV.
Streamlit	Interface : choix des questions, barre de progression, cartes de résultats colorées par décision.

Remarque : ce projet a nécessité plusieurs corrections pour fonctionner sur cette machine — un conflit TensorFlow/protobuf lors de l'import de sentence_transformers (résolu en forçant USE_TF=0), un bug de la bibliothèque uqlm elle-même où l'attribut interne scorer_names restait None par défaut (contourné en passant explicitement la liste de scorers), et un plantage sur l'encodage cp1252 de la console Windows face aux emojis des logs internes d'Agno (résolu en forçant l'UTF-8 sur stdout/stderr).

uq_pipeline.py

Moteur partagé entre le script CLI et l'interface : agent, calcul d'incertitude, génération.

Lignes 8-27

```
import os

os.environ.setdefault("USE_TF", "0")

import pandas as pd
from agno.agent import Agent
from agno.models.groq import Groq
from agno.tools.python import PythonTools
from dotenv import load_dotenv
from langchain_groq import ChatGroq
from uqlm import BlackBoxUQ
from uqlm.utils import load_example_dataset, math_postprocessor

load_dotenv()
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
```

Ligne cruciale placée **avant** tout import d'uqlm : `os.environ.setdefault("USE_TF", "0")`. La bibliothèque `sentence-transformers`, utilisée en interne par `uqlm` pour le score de similarité cosinus, tente par défaut de charger TensorFlow en plus de PyTorch ; sur cette machine, la version de TensorFlow installée est incompatible avec la version de protobuf présente, ce qui faisait planter l'import. Comme seul PyTorch est réellement nécessaire, désactiver TensorFlow évite le problème.

Lignes 29-35

```
INSTRUCTION_MATH = "Lorsque vous résolvez ce problème mathématique, ne retournez que la réponse sans texte supplémentaire.\n"

DEFAULT_SCORERS = ["semantic_negentropy", "noncontradiction", "exact_match", "cosine_sim"]
```

`INSTRUCTION_MATH` est préfixée à chaque question pour que le modèle réponde de façon minimaliste (juste le nombre), ce qui facilite la comparaison entre les réponses générées. `DEFAULT_SCORERS` liste les quatre métriques de cohérence calculées entre les réponses : elle est passée **explicitement** à `BlackBoxUQ` plus loin pour contourner un bug de la bibliothèque — quand `scorers` n'est pas fourni, un attribut interne (`self.scorer_names`) reste à `None` au lieu d'utiliser la liste par défaut, ce qui fait planter le calcul des scores avec `TypeError: argument of type 'NoneType' is not iterable`.

Lignes 37-71

```

agent_hallucination = Agent(
    name="AgentDetectionHallucination",
    role="Analyser les réponses incertaines et proposer une action appropriée.",
    model=Groq(id="openai/gpt-oss-120b", api_key=GR0Q_API_KEY),
    tools=[PythonTools()],
    markdown=False,
    description="""
Vous êtes un agent spécialisé dans l'analyse des réponses générées par un modèle de langage.
Votre rôle est de détecter les réponses avec un risque élevé d'hallucination basé uniquement sur le
score d'incertitude fourni... Vous n'avez pas accès à la vérité terrain...
""",
    instructions="""
Tâche:
Basé uniquement sur la valeur d'incertitude, déterminer l'action la plus appropriée selon ces
règles:
Si incertitude < 0.2 → ACCEPTER
Si 0.2 ≤ incertitude ≤ 0.5 → REFORMULER
Si incertitude > 0.5 → REFUSER

Format de Réponse (strictement requis):
Toujours répondre sur une seule ligne, commençant par l'action en MAJUSCULES, suivi d'une
justification.
""",
)

```

L'agent de décision. Son description insiste bien sur le fait qu'il **n'a pas accès à la vérité terrain** — il ne sait jamais si la réponse est réellement correcte, seulement si elle est *cohérente* d'une génération à l'autre. Les instructions fixent des seuils numériques stricts (0.2 et 0.5) et imposent un format de sortie sur une seule ligne, facile à parser ensuite (decision.split()[0] donne le mot ACCEPTER/REFORMULER/REFUSER).

Lignes 74-93

```

def calculer_incertitude(row):
    """Une entropie_sémantique plus faible = incertitude plus élevée."""
    incertitude_sémantique = 1 - row.get("semantic_negentropy", 0)
    incertitude_correspondance_exacte = 1 - row.get("exact_match", 0)
    incertitude_cosinus = 1 - row.get("cosine_sim", 0)
    incertitude_contradiction = 1 - row.get("noncontradiction", 0)

    incertitude_primaire = incertitude_sémantique
    incertitude_combinee = (incertitude_sémantique + incertitude_correspondance_exacte +
    incertitude_cosinus) / 3

    return incertitude_primaire, {
        "sémantique": incertitude_sémantique, "correspondance_exacte":
    incertitude_correspondance_exacte,
        "cosinus": incertitude_cosinus, "contradiction": incertitude_contradiction, "combinee":
    incertitude_combinee,
    }

```

uqlm fournit des scores de **cohérence** (plus le score est haut, plus les réponses générées se ressemblent). Cette fonction les inverse (1 - score) pour obtenir des scores **d'incertitude** plus intuitifs (plus haut = moins fiable), et calcule une incertitude « combinée » en moyennant trois des quatre métriques. C'est incertitude_primaire (basée uniquement sur l'entropie sémantique) qui est transmise à l'agent de décision.

Lignes 96-98

```

def load_svamp_questions(n: int = 5) -> list[str]:
    svamp = load_example_dataset("svamp", n=n)
    return list(svamp["question"])

```

Charge n questions du jeu de données public SVAMP (des petits problèmes mathématiques d'école primaire), fourni en exemple par la bibliothèque uqlm elle-même.

Lignes 101-133

```

async def analyser_questions(questions: list[str], progress_callback=None) -> pd.DataFrame:
    if progress_callback:
        progress_callback("generation", 0, len(questions))

    prompts = [INSTRUCTION_MATH + q for q in questions]
    llm = ChatGroq(model="openai/gpt-oss-20b", temperature=0)
    bbuq = BlackBoxUQ(llm=llm, scorers=DEFAULT_SCORERS)
    results = await bbuq.generate_and_score(prompts)
    df = results.to_df()

    df["question"] = questions

    if "response" in df.columns:
        df["sortie_traitee"] = df["response"].apply(math_postprocessor)
        ...

    donnees_incertainite = []
    for _, row in df.iterrows():
        incertainite, detail = calculer_incertainite(row)
        donnees_incertainite.append({"incertainite": incertainite, **detail})
    df = pd.concat([df, pd.DataFrame(donnees_incertainite)], axis=1)

```

Le cœur du pipeline. `bbuq.generate_and_score()` fait le plus gros du travail en une seule ligne : pour chaque prompt, elle génère une réponse principale **et plusieurs réponses candidates supplémentaires** (5 par défaut), puis calcule les quatre scores de cohérence entre elles — le tout de façon asynchrone. `df["question"] = questions` est un ajout important : le DataFrame retourné par `uqlm` ne contient que le *prompt complet* (avec `INSTRUCTION_MATH` ajoutée), jamais la question d'origine seule ; sans cette ligne, l'agent de décision recevait toujours « Question : N/A » dans le script d'origine — un bug réel découvert et corrigé lors de la construction de ce module. Les scores d'incertitude sont ensuite calculés pour chaque ligne et fusionnés dans le tableau.

Lignes 135-157

```

decisions = []
for i, row in df.iterrows():
    if progress_callback:
        progress_callback("decision", i + 1, len(df))

    response = row.get("response") or row.get("generation", "")
    incertainite = row.get("incertainite", 1.0)
    contexte_prompt = f"""
Question : {row["question"]}
Réponse : {response}
Incertainite : {incertainite:.3f}
""".strip()

    try:
        reponse_agent = agent_hallucination.run(contexte_prompt)
        decision = reponse_agent.content if hasattr(reponse_agent, "content") else
str(reponse_agent)
    except Exception as e:
        decision = f"ERREUR: {e}"

    decisions.append(decision)

df["decision_agent"] = decisions
return df

```

Pour chaque question, construit un petit prompt texte avec la vraie question, la réponse générée et le score d'incertitude, puis appelle l'agent de décision. Le `try/except` capture toute erreur (y compris un dépassement de quota Groq) sans faire planter le traitement des autres questions — l'erreur est simplement enregistrée comme « décision » pour cette ligne, visible ensuite dans les résultats plutôt que de tout interrompre.

allucination_detect.py

Script CLI : lance le pipeline sur 5 questions SVAMP et affiche/sauvegarde les résultats.

Lignes 1-17

```
import asyncio
import pandas as pd
from uq_pipeline import analyser_questions, load_svamp_questions

async def main():
    questions = load_svamp_questions(n=5)
    print("----- Questions-----")
    for idx, q in enumerate(questions, 1):
        print(f"{idx}. {q}")
```

Ce script réutilise entièrement uq_pipeline.py plutôt que de dupliquer la logique — une simplification par rapport à sa version d'origine, qui contenait toute la logique de génération et d'analyse directement dans ce fichier. Charge 5 questions SVAMP et les affiche.

Lignes 19-31

```
def _progress(etape, i, total):
    if etape == "generation":
        print("\n Génération des réponses et calcul de l'incertitude...")
    elif etape == "decision":
        print(f" Analyse par l'agent {i}/{total}...")

df = await analyser_questions(questions, progress_callback=_progress)

print("\n Colonnes disponibles :", df.columns.tolist())
```

Définit une fonction de callback simple qui affiche la progression dans la console (contrepartie textuelle de la barre de progression graphique utilisée côté Streamlit), puis lance le pipeline complet.

Lignes 33-46

```
for i, row in df.iterrows():
    print(f" Q{i + 1} → Incertitude: {row['incertitude']:.3f} → {row['decision_agent']}")

df.to_csv("resultats_math_uq_corrige.csv", index=False)
print("\n Résultats sauvegardés dans resultats_math_uq_corrige.csv")

decisions = [result.split()[0] for result in df["decision_agent"] if result.split()]
print(pd.Series(decisions).value_counts())

if __name__ == "__main__":
    asyncio.run(main())
```

Affiche le résultat de chaque question, sauvegarde le tableau complet en CSV, puis calcule un petit résumé statistique (combien de ACCEPTER/REFORMULER/REFUSER au total) via pandas.Series.value_counts().

Interface Streamlit : choix des questions, barre de progression, cartes colorées par décision.

Lignes 1-16

```
import asyncio
import sys

for _stream in (sys.stdout, sys.stderr):
    if hasattr(_stream, "reconfigure"):
        _stream.reconfigure(encoding="utf-8", errors="replace")

import streamlit as st
from uq_pipeline import GROQ_API_KEY, analyser_questions, load_svamp_questions
```

Avant même d'importer Streamlit, reconfigure l'encodage de la sortie standard en UTF-8. Sur Windows, la console du serveur Streamlit utilise par défaut l'encodage cp1252, incapable d'afficher les emojis que la bibliothèque Agno écrit dans ses logs internes (lors des appels d'outils) — sans ce correctif, `agent.run()` plantait avec une `UnicodeEncodeError`, observé lors des tests.

Lignes 18-27

```
DECISION_STYLE = {
    "ACCEPTER": ("success", ""),
    "REFORMULER": ("warning", ""),
    "REFUSER": ("error", ""),
}

if "results_df" not in st.session_state:
    st.session_state.results_df = None
```

Table de correspondance entre chaque décision possible et à la fois une couleur Streamlit (`st.success/st.warning/st.error`) et un emoji, utilisée plus loin pour styliser chaque carte de résultat de façon cohérente.

Lignes 29-47

```
with st.sidebar:
    source = st.radio("Source des questions", ["Exemples SVAMP (maths)", "Mes propres questions"])

    if source == "Exemples SVAMP (maths)":
        n_questions = st.slider("Nombre de questions", 1, 10, 3)
        custom_questions = None
    else:
        custom_text = st.text_area("Questions", height=150, placeholder="Combien font 12 + 7 ?\n...")
        custom_questions = [q.strip() for q in custom_text.splitlines() if q.strip()]

    launch = st.button(" Lancer l'analyse", use_container_width=True)
```

Deux modes : questions d'exemple SVAMP (avec un curseur pour en choisir le nombre, de 1 à 10) ou questions personnalisées saisies librement, une par ligne. C'est un ajout par rapport au script CLI d'origine, qui ne testait que le jeu de données SVAMP.

Lignes 49-70

```

if launch:
    questions = load_svamp_questions(n=n_questions) if source == "Exemples SVAMP (maths)" else
    (custom_questions or [])

    if not questions:
        st.warning(" Ajoute au moins une question avant de lancer l'analyse.")
    else:
        progress_bar = st.progress(0.0, text=" Génération des réponses et calcul de
l'incertitude...")

        def _on_progress(etape, i, total):
            if etape == "generation":
                progress_bar.progress(0.15, text="...")
            elif etape == "decision":
                progress_bar.progress(0.15 + 0.85 * (i / total), text=f" Décision de l'agent
({i}/{total})...")

        try:
            df = asyncio.run(analyser_questions(questions, progress_callback=_on_progress))
            st.session_state.results_df = df
            progress_bar.empty()
        except Exception as e:
            progress_bar.empty()
            st.error(f" Erreur pendant l'analyse : {e}")

```

La progression est répartie en deux phases visuelles : les 15 premiers % représentent la génération des réponses (une seule étape rapide à signaler), puis les 85% restants sont proportionnels au nombre de questions déjà tranchées par l'agent de décision. `asyncio.run()` est nécessaire ici car `analyser_questions()` est une fonction async, alors que Streamlit exécute son script de façon synchrone.

Lignes 77-107

```

decisions = [d.split()[0] if d.split() else "?" for d in df["decision_agent"]]
counts = {k: decisions.count(k) for k in ("ACCEPTER", "REFORMULER", "REFUSER")}

col1, col2, col3, col4 = st.columns(4)
col1.metric(" Questions analysées", len(df))
col2.metric(" Acceptées", counts["ACCEPTER"])
...

for i, row in df.iterrows():
    decision_word = row["decision_agent"].split()[0] if row["decision_agent"].split() else "?"
    style, emoji = DECISION_STYLE.get(decision_word, ("info", ""))

    with st.expander(f"{emoji} Q{i + 1} – {row['question'][:80]}..."):
        st.markdown(f"*** Question : ** {row['question']}")
        st.markdown(f"*** Réponse du modèle : ** {row.get('response', row.get('generation', ''))}")

        m1, m2, m3, m4, m5 = st.columns(5)
        m1.metric("Incertitude", f"{row['incertitude']:.2f}")
        ...
        getattr(st, style)(f"***{row['decision_agent']}***")

    with st.popover(" Voir les réponses échantillonnées"):
        for j, s in enumerate(row.get("sampled_responses") or [], 1):
            st.markdown(f"{j}. {s}")

```

Compte les décisions par type pour les métriques du haut, puis affiche une carte repliable par question : question complète, réponse du modèle, les 5 scores détaillés côte à côte, et la décision affichée avec la bonne couleur grâce à `getattr(st, style)(...)` (appelle dynamiquement `st.success`, `st.warning` ou `st.error` selon `DECISION_STYLE`). Un `st.popover` permet de dérouler, à la demande, les réponses individuelles échantillonnées qui ont servi au calcul d'incertitude — utile pour comprendre *pourquoi* une question a été jugée incertaine.